

ENIAD TP4

POO en Java

Partie 1 : Gestion des exceptions

Exercice 1: Gestion d'une Division par Zéro

Écrire un programme Java qui demande à l'utilisateur de saisir deux nombres, puis effectue une division entre ces deux nombres. Gérer l'exception de division par zéro (`ArithmeticException`) en affichant un message approprié si le deuxième nombre est zéro.

Exemple d'exécution :**Entrée :**

Saisissez le premier nombre : 10

Saisissez le deuxième nombre : 0

Sortie :

Erreur: Division par zéro est impossible.

Exercice 2 : Validation de l'Entrée Utilisateur

Écrire un programme Java qui demande à l'utilisateur de saisir un nombre entre 1 et 100. Si l'utilisateur saisit un nombre qui n'est pas dans cette plage, le programme doit afficher un message d'erreur et redemander la saisie.

Exemple d'exécution :**Entrée :**

Saisissez un nombre entre 1 et 100 : 150

Sortie :

Erreur: Le nombre doit être compris entre 1 et 100.

Saisissez un nombre entre 1 et 100 : 50

Sortie :

Vous avez saisi : 50

Exercice 3 : Conversion de Chaîne en Entier

Écrire un programme Java qui demande à l'utilisateur de saisir une chaîne de caractères représentant un nombre entier. Le programme doit convertir cette chaîne en un entier et afficher le résultat. Gérer l'exception `NumberFormatException` si la chaîne entrée ne peut pas être convertie en un entier.

Exemple d'exécution :**Entrée :**

Entrez un nombre entier : abc

Sortie :

Erreur: Entrée invalide. Veuillez entrer un nombre entier.

Exercice 4 : Vérification de l'Âge

Écrire une classe en Java pour représenter une personne avec un attribut d'âge. La classe devrait avoir une méthode pour définir l'âge de la personne. Si l'âge spécifié est négatif, la méthode devrait lever une exception `IllegalArgumentException`.

Exemple d'exécution :

Sortie :

Erreur: L'âge ne peut pas être négatif.

Exercice 5 : Gestion des Produits

Créez une application de gestion de produits avec une classe `Produit` qui a deux attributs : `nom` et `prix`. Lors de la création d'un produit, si le prix est négatif, une exception spécifique nommée `PrixNegatifException` doit être lancée.

Dans cet exercice, il faut créer votre spécifique class d'exception.

Exercice 6 : Gestion d'un Compte Bancaire

Créez une application de gestion de comptes bancaires avec une classe `CompteBancaire` qui a un attribut `solde`. Lorsqu'un retrait est effectué sur le compte et que le solde devient négatif, une exception spécifique nommée `SoldeNegatifException` doit être lancée.

Dans cet exercice, il faut créer votre spécifique class d'exception.

Partie 1 : Les collections

Exercice 1 : Gestion des contacts

Créez une classe en Java nommée `GestionContacts` qui permet de gérer une liste de contacts. Chaque contact est représenté par un nom et un numéro de téléphone. La classe doit permettre d'ajouter des contacts, de les supprimer, de les afficher et de rechercher un contact par son nom.

Exemple d'exécution :

Sortie

Liste des contacts :

Nom : Alice, Numéro : 1234567890

Nom : Bob, Numéro : 9876543210

Nom : Charlie, Numéro : 5555555555

Après suppression de Bob :

Liste des contacts :

Nom : Alice, Numéro : 1234567890

Nom : Charlie, Numéro : 5555555555

Recherche de Charlie :

Nom : Charlie, Numéro : 5555555555

Exercice 2 : Gestion des membres d'un club

Imaginez un club sportif qui souhaite gérer ses membres. Créez une classe en Java nommée `GestionMembresClub` qui permet d'ajouter des membres au club et d'afficher la liste des membres uniques.

Exemple d'exécution :

Sortie

Membres du club :

Bob

Charlie

Alice

Exercice 3 : Gestion des Produits dans un Magasin

Créez une classe en Java nommée `GestionProduitsMagasin` qui permet de gérer les produits disponibles dans un magasin. Chaque produit est représenté par un identifiant unique (clé) et un nom (valeur).

Exemple d'exécution :

Sortie

Produits disponibles :

ID : 1, Nom : Ordinateur portable

ID : 2, Nom : Smartphone

ID : 3, Nom : Tablette

Après suppression :

Produits disponibles :

ID : 1, Nom : Ordinateur portable

ID : 3, Nom : Tablette